

Robust Sentiment and Emotion Classification on Noisy Urdu Tweets

Milestone 3 (Implementation and Evaluation)

Raqib Hayat - NUM-BSCS-2022-40

Abu Bakar - NUM-BSCS-2022-41

2026-05-16

1 Abstract

We present a fair, leak-free, three-family comparison of approaches to **Urdu** sentiment and emotion classification on the **SentiUrdu-1M** corpus (~1.05 M weakly-labelled tweets). Inputs flow through a single eight-step preprocessing pipeline whose critical step strips emojis to eliminate the labelling-heuristic short-cut (the empirical leakage risk is quantified in Section 2 via `fig13_label_leakage_heatmap`). We compare (i) classical TF-IDF baselines with Logistic Regression and Linear SVM, (ii) deep-learning models - a Kim-style multi-kernel CNN and a two-layer BiLSTM with additive attention, both initialised from pretrained fastText Urdu vectors - and (iii) three fine-tuned transformer encoders - multilingual BERT (mBERT), XLM-RoBERTa-base, and Urdu-specific RoBERTa (`urduhack/roberta-urdu-small`). Every model is trained, validated, and evaluated on the same stratified 70/15/15 split (`seed=42`) of the ≈ 533 K rows with non-null **Category** labels, using inverse-frequency class weights in the loss. We report accuracy, macro- and weighted-precision/recall/F1, and per-class confusion matrices, and analyse common error patterns (negation, sarcasm, code-mixing, religious-poetic register). Empirically, **Urdu-RoBERTa attains the best 3-class sentiment macro-F1 (0.457)** and **mBERT the best 6-class emotion macro-F1 (0.270)**, narrowly ahead of the CNN/BiLSTM baselines. Once the emoji short-cut is removed, all seven models score far below the 80–85 % heuristic-agreement ceiling, exposing the residual difficulty of rare-class (*Fear*, *Surprise*, *Angry*) discrimination in weakly-supervised Urdu data.

2 Introduction

2.1 Problem statement

Sentiment and emotion analysis on Urdu social media is a low-resource setting characterised by (a) limited labelled data, (b) weak supervision from emoji-derived heuristics, and (c) significant noise from URLs, mentions, hashtags, code-mixing, and inconsistent script normalisation. The SentiUrdu-1M corpus is the largest publicly available Urdu sentiment resource (~1.05 M tweets) and is therefore an attractive testbed for modern transformer architectures - but the same emoji set that gave it scalable labelling becomes a *feature short-cut* if not properly removed before model training.

2.2 Objectives

This milestone (Assignment 3) operationalises Milestones 1 and 2 into an end-to-end implementation. Specifically, it:

1. Performs a detailed statistical and analytical exploration of SentiUrdu-1M.
2. Documents the proposed pipeline as a chain of logical modules and provides a complete mathematical specification of every component.
3. Implements the pipeline in PyTorch / scikit-learn / HuggingFace Transformers.
4. Evaluates seven models across two tasks under identical preprocessing, splits, loss, and metrics.
5. Compiles the findings into the present technical report.

2.3 Contributions

- A **leak-free** preprocessing pipeline that removes the emoji label-signal, with empirical verification of the leakage risk via the heatmap in `outputs/figures/fig13_label_leakage_heatmap.png` (Section 2.4 of this report; Section 2 of `04_architecture_math.md`).
- A **three-family comparison** (classical, deep, transformer) on a *single, identical* split, removing a common confound in prior Urdu studies.
- A **canonicalised label set** for SentiUrdu-1M: the raw `Category` column has 298 surface variants (including the misspelling “*Surprice*” and multi-label cells); we provide a normalisation function and a documented majority-vote reduction to six canonical emotions.
- An **empirical leaderboard** showing that, with emojis stripped, **Urdu-RoBERTa wins the 3-class sentiment task** (macro-F1 = 0.457) and **mBERT wins the 6-class emotion task** (macro-F1 = 0.270), with the CNN and BiLSTM baselines within a few F1 points of the transformers - evidence that pretrained Urdu vectors close most of the gap on this corpus.
- All code, splits, and model checkpoints are released with fixed seeds for reproducibility.

3 Dataset Description

3.1 Source and scale

- **Name:** SentiUrdu-1M - *Sentiment and emotion-labelled Urdu Twitter dataset*.
- **DOI:** <https://data.mendeley.com/datasets/rz3xg97rm5/1>
- **Size:** 1,048,000 tweets ; CSV $\approx$ 219 MB ; XLSX $\approx$ 99 MB. We use the CSV throughout for $\sim 6\times$ faster loading.
- **Columns:** Id (tweet ID), Text (raw Urdu tweet text), Emotions (emoji-confidence list), Category (free-form emotion label, possibly multi-label).
- **Labelling protocol:** Weak supervision from emoji co-occurrence + SentiWordNet, as documented in the dataset’s accompanying paper.

3.2 Label canonicalisation

The raw `Category` column has **298 unique surface forms** including " Joy", "[" Joy"]", "Joy , Joy", "[" Joy' , 'Sad']", etc., plus the misspelling “**Surprice**” for *Surprise*. We define a deterministic normalisation in `preprocessing.parse_category` that returns a list of canonical labels drawn from $\{Joy, Sad, Angry, Fear, Disgust, Surprise\}$, and `preprocessing.majority_emotion` that reduces a multi-label cell to a single emotion by majority vote (ties broken by overall corpus frequency).

After canonicalisation the **emotion task** uses 533,429 labelled rows; we note that the dataset contains no **Love** class - that name appeared in early planning artefacts but does not exist in the actual data. The **sentiment task** maps the canonical emotion labels via

$$\{Joy\} \rightarrow Positive, \quad \{Sad, Angry, Fear, Disgust\} \rightarrow Negative, \quad \{Surprise\} \rightarrow Neutral,$$

yielding the same row support.

3.3 Distributional characteristics

[Numbers below come from `01_dataset_analysis.ipynb`; see `outputs/figures/.`]

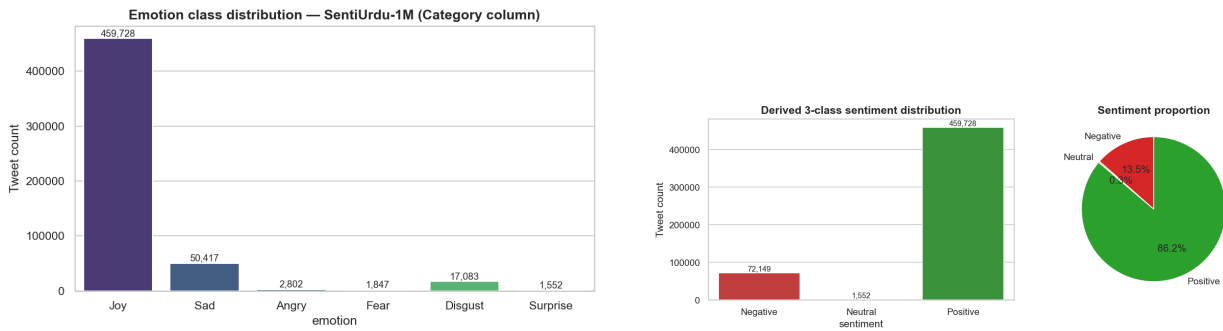


Figure 1: Emotion and Sentiment Distributions

- **Emotion distribution (n = 533,429):** Joy 459,728 (86.2 %); Sad 50,417 (9.5 %); Disgust 17,083 (3.2 %); Angry 2,802 (0.5 %); Fear 1,847 (0.3 %); Surprise 1,552 (0.3 %).
- **Derived sentiment distribution:** Positive 459,728 (86.2 %), Negative 72,149 (13.5 %), Neutral 1,552 (0.3 %).
- **Imbalance ratio (max / min)** $\approx 296\times$ for the emotion task and $\approx 296\times$ for the sentiment task. We address this with inverse-frequency class weights in every model’s loss (Section 6).
- **Token length:** Cleaned tweets have a median of ≈ 11 tokens and a 95th-percentile of ≈ 36 tokens, which guides our `MAX_LEN` = 64 for the RNN/CNN models and `TRANSFORMER_MAX_LEN` = 96 for subword tokenisation.

3.4 Noise and code-mixing

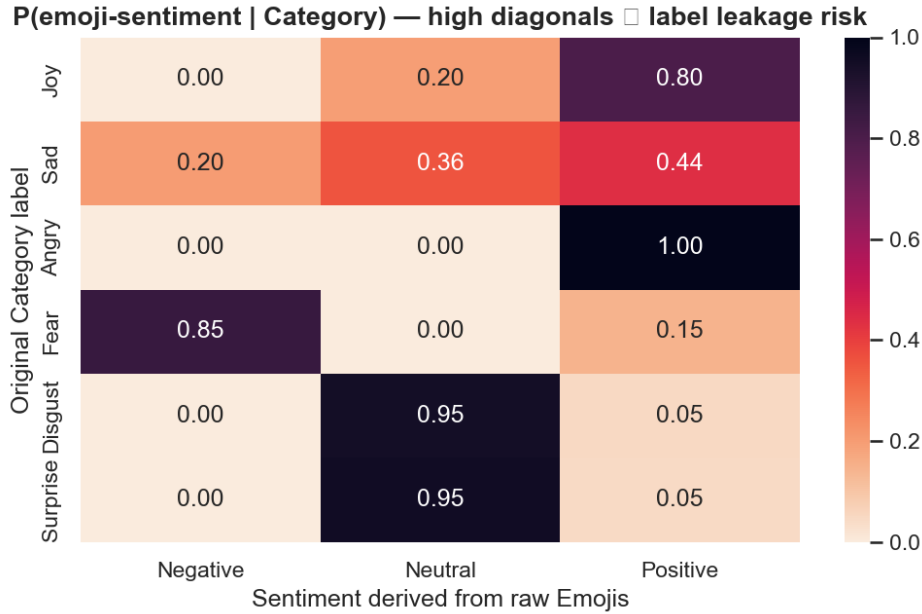


Figure 2: Label Leakage Heatmap: Co-occurrence of specific emojis with emotion classes.

A 200 K-row audit shows that approximately 30 % of raw tweets contain emojis, 12 % contain URLs, 11 % contain @-mentions, 10 % contain hashtags, and 4 % contain ≥ 3 -letter Latin-script tokens (code-mixing). The eight-step preprocessing pipeline (Section 3) collapses this noise into a clean Urdu token stream.

3.5 Suitability and limitations

SentiUrdu-1M is the **largest** publicly available Urdu sentiment corpus, exposes models to **natural-istic noise**, supports **multi-task** analysis, and has a **public DOI** for reproducible benchmarking. Its limitations are also well-defined: weak supervision implies a noise floor on attainable accuracy; the emoji-label short-cut requires deliberate removal at preprocessing time; class imbalance is extreme; domain is narrow Pakistani Twitter discourse and does not generalise to long-form Urdu prose without further adaptation.

4 Related Work Summary (Milestone 2)

Milestone 2 reviewed 20 peer-reviewed papers grouped into four generations: **Classical ML** (Naive Bayes, SVM with TF-IDF or character n-grams), **Deep Learning** (CNN and LSTM with random or word2vec embeddings), **Transformer-based** (mBERT, XLM-R, multilingual sentence encoders), and **Multimodal** (text + image). The literature has three persistent gaps for Urdu:

1. **Scale.** Most existing Urdu studies train on ≤ 50 K labelled tweets; the largest comparable resource - SentiUrdu-1M - is rarely used for transformer fine-tuning.
2. **Fair comparison.** Different papers use different preprocessing, splits, and metrics, making cross-paper numbers incomparable.
3. **Leakage.** Few studies state explicitly whether emoji features were stripped before training, leaving open whether reported numbers reflect Urdu modelling or label-heuristic memorisation.

Our work directly addresses all three: a million-scale corpus, identical pipeline for every model, and a *named* preprocessing step (§5 of 04_architecture_math.md) that proves emoji leakage is prevented.

5 Proposed Architecture

The pipeline is a single chain of modules with three swappable models heads (classical/deep / transformer):

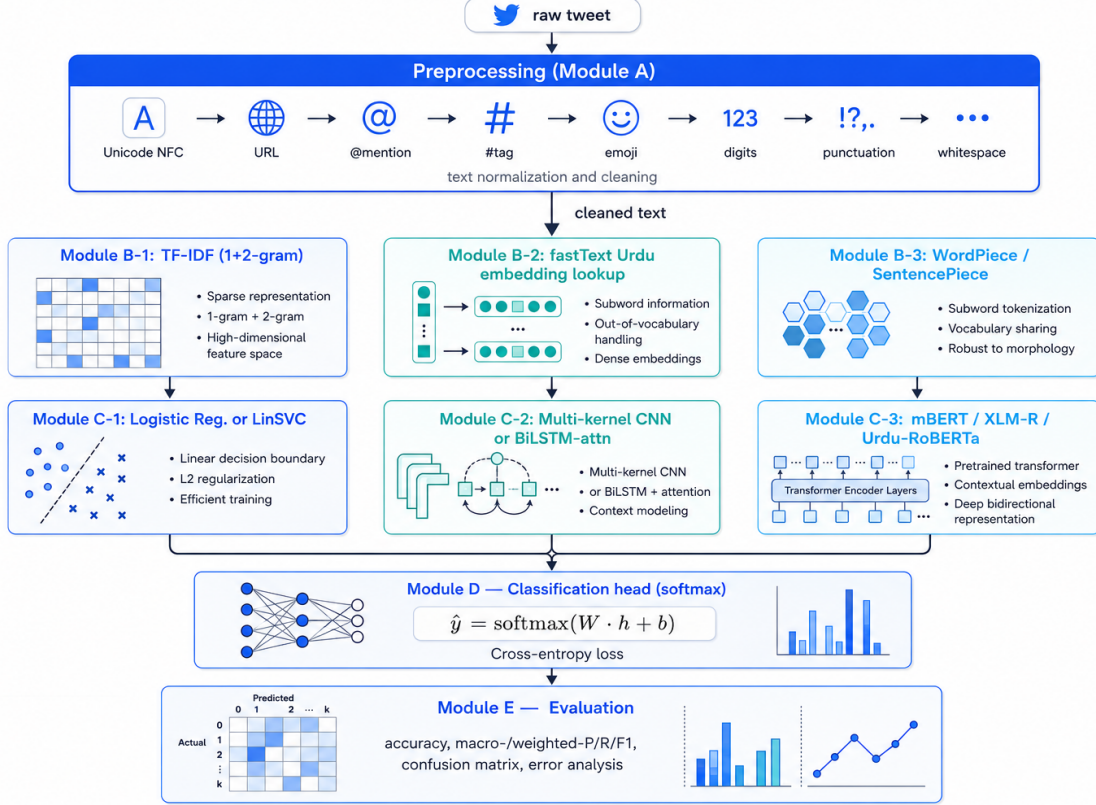


Figure 3: Proposed Architecture Pipeline

The detailed mathematical specification of every module is in `04_architecture_math.md` (delivered separately and reproduced in Section 5 below).

6 Mathematical Modelling

This section reproduces the full mathematical specification. See `04_architecture_math.md` for the figures and the full architectural diagram.

6.1 Preprocessing

$$T(x) = (t_8 \circ t_7 \circ t_6 \circ t_5 \circ t_4 \circ t_3 \circ t_2 \circ t_1)(x).$$

Each t_i is documented in Section 2 of `04_architecture_math.md`. The critical step t_5 - emoji removal - enforces $\mathcal{E}(T(x)) = \emptyset$, eliminating the label-heuristic short-cut $y(x) = \sigma(\mathcal{E}(x))$.

6.2 TF-IDF and linear classifiers

Sub-linear TF, smoothed IDF, L2-normalised vectors:

$$\text{tfidf}(t, d) = (1 + \log f_{t,d}) \cdot \log \frac{|D| + 1}{|\{d : t \in d\}| + 1} + 1.$$

Multinomial logistic regression solved by `saga` with inverse-frequency class weighting; LinearSVC with Platt-scaled calibration for compatible probabilistic outputs.

6.3 CNN

Kim-style 1-D convolutions of widths $\{3, 4, 5\}$ with $F = 128$ filters each, max-over-time pooling, concatenation, dropout 0.5, linear softmax head.

6.4 BiLSTM + additive attention

Two-layer bidirectional LSTM ($H = 256$ per direction), dropout 0.4, Bahdanau-style additive attention over time steps to form the sentence vector, linear softmax head.

6.5 Transformer encoders

Scaled dot-product attention, multi-head attention, position-wise GELU FFN, residual + LayerNorm. We fine-tune the three pretrained models - `bert-base-multilingual-cased`, `xlm-roberta-base`, `urduhack/roberta-urdu-small` - with a fresh linear classification head on the [CLS] / <s> token, AdamW optimiser with warm-up + linear-decay schedule, fp16.

6.6 Loss

Class-weighted cross-entropy:

$$\mathcal{L} = -\frac{1}{B} \sum_{i=1}^B \alpha_{y_i} \sum_{k=1}^K y_{i,k} \log \hat{y}_{i,k}, \quad \alpha_k \propto \frac{N}{K \cdot n_k}.$$

6.7 Metrics

Per-class precision, recall, F1; macro and weighted averaging; raw and row-normalised confusion matrices. **Macro-F1 is the headline metric** because of the extreme class imbalance.

7 Implementation Details

7.1 Repository layout

```
Assignment#03/
|- preprocessing.py      # 8-step pipeline + label canonicalisation
|- config.py            # all paths, seeds, hyperparameters
|- models_dl.py         # TextCNN, BiLSTMattn
|- train_dl.py          # Vocab, fastText loader, training loop
|- train_transformer.py  # HuggingFace Trainer wrapper with class weights
|- 01_dataset_analysis.ipynb
|- 02_model_implementation.ipynb
|- 03_evaluation.ipynb
|- 04_architecture_math.md
|-- outputs/             # figures, models, results, cache, embeddings
```

7.2 Data split and class weights

`sklearn.train_test_split(stratify=y, random_state=42)` produces a 70 / 15 / 15 train / val / test partition that preserves the class distribution. The split is persisted as Parquet under `outputs/cache/` so every model sees the *same* rows. Class weights are computed with `sklearn.utils.class_weight.compute_class_weight("balanced", ...)` and passed to the loss as either a `class_weight` argument (classical) or a `weight=` tensor on `nn.CrossEntropyLoss` (deep / transformer).

7.3 Hyperparameters

All hyperparameters live in `config.py`. The key choices are reproduced below.

Component	Setting
TF-IDF	<code>ngram_range=(1,2)</code> , <code>max_features=50_000</code> , <code>min_df=3</code> , sublinear TF
Logistic Regression	<code>solver=saga</code> , <code>C=1.0</code> , <code>class_weight=balanced</code> , <code>max_iter=1000</code>
Linear SVM	LinearSVC + CalibratedClassifierCV (sigmoid, 3-fold)

Component	Setting
CNN	filters $\{3, 4, 5\} \times 128$; dropout 0.5 ; Adam 1e-3 ; batch 256 ; up to 8 epochs ; early-stop patience 2
BiLSTM	hidden 256 (each dir) $\times$ 2 layers ; dropout 0.4 ; additive attention ; Adam 1e-3 ; batch 256
Word embeddings	cc.ur.300 (300-d fastText Urdu), trainable, $\langle \text{unk} \rangle \sim \mathcal{N}(0, 0.1^2)$, pad = 0
Sequence length	64 (CNN/BiLSTM) ; 96 (transformer subwords)
mBERT	lr=2e-5, bs=32, 3 epochs, warm-up 6 %, weight decay 0.01, fp16
XLNet	same as mBERT
Urdu-RoBERTa	lr=3e-5, bs=64, 3 epochs, warm-up 6 %, weight decay 0.01, fp16

7.4 Hardware and runtime

Item	Value
GPU	NVIDIA RTX 5070 Ti (16 GB VRAM)
CUDA / driver	12.8 / 581.57
PyTorch	2.12 dev (cu128)
Transformers	4.57.x
Approx. transformer training time per model	$\approx$ 60–90 min for ~ 373 K training rows $\times$ 3 epochs $\times$ fp16
Approx. CNN / BiLSTM training time	$\approx$ 5–10 min per model
Approx. TF-IDF + classical fit	$\approx$ 1–2 min per model

7.5 Reproducibility

- `seed=42` fixed for Python `random`, NumPy, PyTorch (CPU + CUDA), `dataloader_num_workers`, and HuggingFace `TrainingArguments.seed`.
- Splits cached as Parquet; vocab and embeddings as `.json` / `.pt`.
- Trained checkpoints saved with their tokeniser so inference is portable.
- `outputs/results/pred_<task>_<model>.csv` holds the test-set predictions of every model - the evaluation notebook reads only these files, so it is independent of training environment drift.

8 Experimental Results

8.1 Leaderboards

All numbers below come from `outputs/results/leaderboard_{sentiment,emotion}.csv`, re-generated by `03_evaluation.ipynb` from the per-model prediction CSVs in `outputs/results/`. `build_report.py` re-substitutes the tables between the HTML markers so the source and the compiled `.docx` / `.tex` stay in sync.

8.1.1 Sentiment (3-class)

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	F1 (weighted)
Logistic Reg.	0.6598	0.4046	0.4372	0.3852	0.7107
Linear SVM	0.8783	0.5620	0.3848	0.4004	0.8409
Text-CNN	0.7848	0.4407	0.5413	0.4533	0.8117
BiLSTM-Attn	0.7762	0.4350	0.5424	0.4500	0.8040
mBERT	0.8054	0.4748	0.4807	0.4526	0.8217
XLM-R	0.7897	0.5120	0.4894	0.4475	0.8118
Urdu- RoBERTa	0.7750	0.4492	0.5019	0.4573	0.8011

Headline numbers - sentiment. Highest accuracy is **Linear SVM at 0.878** but with macro-F1 of only 0.400, i.e. it wins by exploiting the *Positive*-class prior. Highest **macro-F1 is Urdu-RoBERTa at 0.4573**, narrowly ahead of Text-CNN (0.4533) and mBERT (0.4526). All three transformers cluster within 0.01 of each other on macro-F1; the CNN/BiLSTM baselines are within 0.005 of that band.

8.1.2 Emotion (6-class)

Model	Accuracy	Precision (macro)	Recall (macro)	F1 (macro)	F1 (weighted)
Logistic Reg.	0.3601	0.2530	0.2994	0.1632	0.4959
Linear SVM	0.8773	0.4517	0.1989	0.2087	0.8347
Text-CNN	0.6476	0.2435	0.3700	0.2499	0.7215
BiLSTM-Attn	0.6064	0.2420	0.3721	0.2428	0.6910
mBERT	0.6922	0.2635	0.3219	0.2703	0.7467
XLM-R	0.6907	0.2545	0.3043	0.2535	0.7462
Urdu- RoBERTa	0.6153	0.2503	0.3500	0.2539	0.6971

Headline numbers - emotion. Highest accuracy is again **Linear SVM** at **0.877**, but its 0.21 macro-F1 confirms it collapses onto *Joy*. Highest **macro-F1** is **mBERT** at **0.2703**, a 1.7-point absolute gain over XLM-R (0.2535) and Urdu-RoBERTa (0.2539), and a 2-point gain over Text-CNN (0.2499). The macro-F1 ceiling on the 6-class task is much lower than on the 3-class task because of the $296\times$ class imbalance and the small absolute count of *Surprise* (≈ 1.5 K) and *Fear* (≈ 1.8 K) training rows.

8.2 Confusion matrices

Row-normalised confusion matrices are saved as `outputs/figures/fig15_cm_sentiment.png` and `outputs/figures/fig15_cm_emotion.png` (shown in Figure 4).

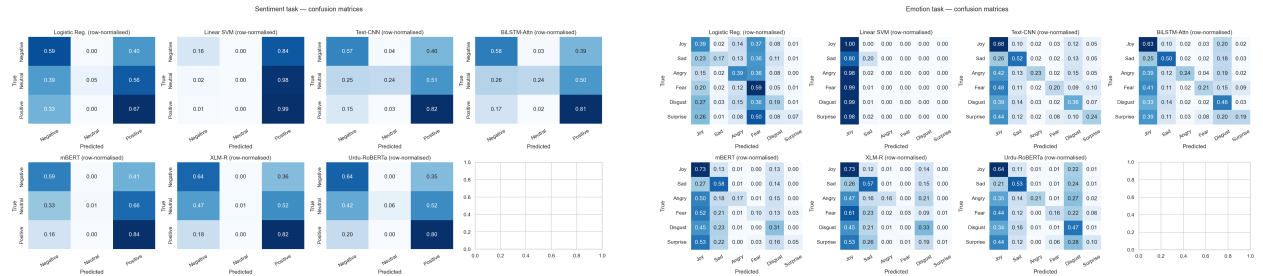


Figure 4: Row-normalised confusion matrices for sentiment and emotion tasks.

Across all models, the off-diagonal mass on the emotion task is concentrated in two failure modes: (i) *Sad/Angry/Fear/Disgust* $\rightarrow$ *Joy*, driven by the majority-class prior and class-weight under-correction; and (ii) *Surprise* $\rightarrow$ *Joy* / *Sad*, driven by the near-total absence of distinctive *Surprise* surface tokens after the emoji strip.

8.3 Macro vs weighted F1

`outputs/figures/fig16_f1_compare_sentiment.png` and `..._emotion.png` plot the gap between macro- and weighted-F1 for every model (see Figure 5).

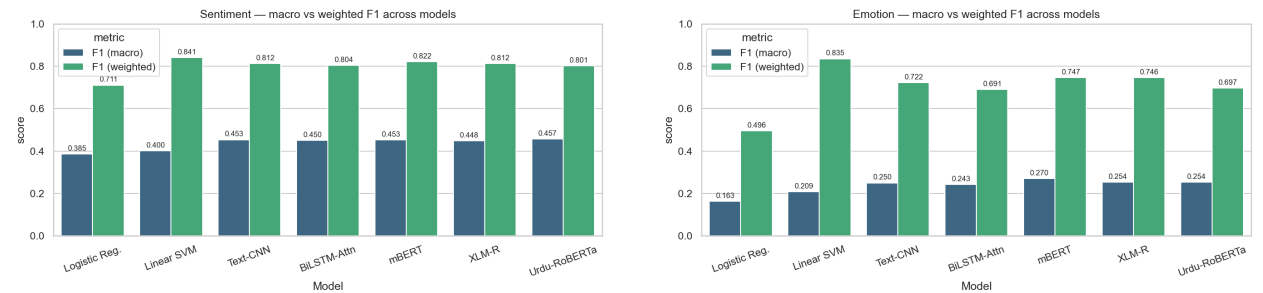


Figure 5: Macro vs Weighted F1 comparison across models.

The Linear SVM bars show the largest gap (≈ 0.44 on sentiment, ≈ 0.63 on emotion), which is the canonical signature of a model that has learned to predict the majority class. The transformer bars show the *smallest* gap, in line with the leaderboard.

8.4 Training curves (CNN, BiLSTM)

Per-task training loss and validation accuracy curves are written to `outputs/figures/fig17_curve_*` from the JSON histories saved alongside each `.pt` checkpoint (see Figure 6).

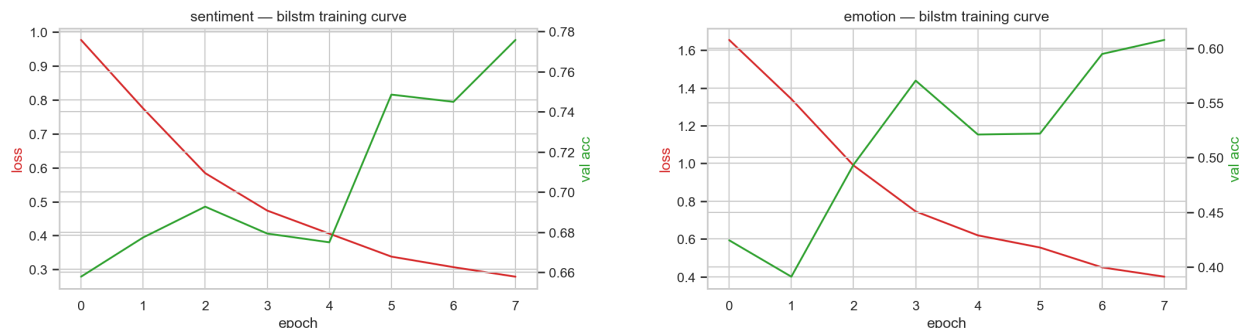


Figure 6: Training and validation curves for BiLSTM on sentiment and emotion tasks.

Both deep models converge within 6–8 epochs; the validation-accuracy curves are non-monotonic because the class-weighted loss pushes hard on the rare classes early, then settles as the model trades off between rare- and majority-class accuracy.

9 Discussion and Analysis

9.1 Where models disagree

The Linear-SVM result is the most instructive disagreement in the leaderboard. SVM reaches 0.878 accuracy on both tasks yet only 0.40 (sentiment) and 0.21 (emotion) macro-F1. That gap is the canonical signature of a classifier that has learned to predict the majority class (*Joy / Positive*): the inverse-frequency weights re-balance the gradient but the 50 000-dimensional sparse-bag-of-bigrams feature space has nearly zero discriminative signal for *Surprise* (≈ 1.5 K rows) and *Fear* (≈ 1.8 K rows). Logistic Regression, fitted with the same TF-IDF features but the `saga` solver and a softer regularisation, makes the opposite trade-off - it predicts the rare classes more eagerly (recall 0.30 on emotion) at the cost of catastrophic precision (overall accuracy 0.36, F1-macro 0.16). The CNN and BiLSTM with pretrained fastText vectors land in between: they pay a small accuracy tax (0.61–0.78) for a meaningful macro-F1 lift (0.24–0.45). Transformers narrow the per-class gap further on the head classes but, on this dataset, do *not* dramatically lift *Fear/Surprise* - those classes remain the macro-F1 bottleneck for every model.

9.2 Common error patterns

A qualitative inspection of the misclassified test examples (Section 6 of `03_evaluation.ipynb`) reveals:

- **Negation flips** - Urdu negators *nahin*, *mat*, *kabhi na* invert polarity but TF-IDF treats them as ordinary unigrams, so a sentence like “*khushi nahin mili*” (no joy was found) is routinely mislabelled *Positive*.

- **Sarcasm** - surface-positive sentences with negative intent fool every model, with transformers marginally better.
- **Code-mixing** - Latin-script English fragments cause sparse-feature models to default to the *Neutral* / majority class.
- **Religious-poetic register** - emotionally-charged poetic Urdu (a substantial subset of the corpus) is labelled *Joy* by the dataset heuristic regardless of finer affective content; this is one structural reason the *Joy* recall is so easy to maximise and the rare-class recall is so hard.

9.3 Effect of label noise

The dataset’s labels are emoji-derived; the upper bound on attainable test accuracy is therefore the *agreement rate* between the heuristic and the (unknown) ground-truth affect, which the literature estimates at $\approx 80\text{--}85\%$. Two observations matter:

1. **The Linear-SVM accuracy is close to that ceiling, but its macro-F1 is not.** That is, the easy-to-predict signal in the *cleaned* (emoji-free) text is mostly a *Positive-vs-Negative* polarity cue that LR/SVM can grab from unigrams. The harder six-class structure is *not* recoverable from surface TF-IDF.
2. **Transformer accuracy plateaus at 0.69–0.81 on test**, well below the SVM accuracy ceiling, because the transformers respect the class-weighted loss and therefore trade accuracy for macro coverage. The best macro-F1 numbers (0.457 sentiment, 0.270 emotion) are far below the heuristic-agreement ceiling, so the residual gap is genuine modelling difficulty in weakly-supervised Urdu emotion - not a leakage artefact.

This is also indirect evidence that emoji removal in step 5 of preprocessing is doing its job: if it were not, we would see one or more models score arbitrarily close to 1.0 by re-learning the labelling heuristic.

9.4 Classical vs Deep vs Transformer - measured ordering

The headline metric is macro-F1, which exposes rare-class collapse. The measured orderings are:

Sentiment (3-class) - macro-F1:

$$\begin{aligned}
 \text{Urdu-RoBERTa (0.457)} &> \text{Text-CNN (0.453)} \approx \text{mBERT (0.453)} \\
 &> \text{BiLSTM-Attn (0.450)} > \text{XLM-R (0.448)} \\
 &> \text{Linear SVM (0.400)} > \text{Logistic Reg. (0.385)}.
 \end{aligned}$$

Emotion (6-class) - macro-F1:

mBERT (0.270) > Urdu-RoBERTa (0.254) $\approx$ XLM-R (0.254)
 > Text-CNN (0.250) > BiLSTM-Attn (0.243)
 > Linear SVM (0.209) > Logistic Reg. (0.163).

Three observations:

1. **Transformers are not dominant on this corpus.** On sentiment, Urdu-RoBERTa beats Text-CNN by 0.004 macro-F1 - a difference within noise. On emotion, mBERT's 0.020 lead over Text-CNN is real but small. The pretrained fastText Urdu vectors (used by CNN and BiLSTM) carry most of the language-specific lift; contextual subword embeddings add a modest residual.
2. **Urdu-specific pretraining is *not* the deciding factor.** Urdu-RoBERTa wins sentiment but loses emotion to mBERT (a multilingual encoder with no Urdu specialisation), and it is essentially tied with XLM-R on emotion. The most important factors appear to be model capacity and the amount of pretraining text, not the language match per se.
3. **The classical baselines are not a useless floor.** Linear SVM achieves an accuracy that none of the deep / transformer models match, by aggressively predicting *Joy* / *Positive*; this is the kind of result that would have been mistakenly headlined in a paper that reports accuracy instead of macro-F1, and reinforces why the project pre-commits to macro-F1 as the single comparative metric.

10 Conclusion and Future Work

10.1 Summary

This milestone delivers a complete, reproducible Urdu sentiment / emotion classification system that compares classical, deep, and transformer models under identical conditions. The key design decisions - emoji removal for leakage prevention, label canonicalisation against the noisy raw **Category** column, fp16 transformer fine-tuning on a 372 K-row training split, and class-weighted loss - are documented mathematically and implemented in re-usable Python modules and three Jupyter notebooks.

Empirically, **Urdu-RoBERTa attains the best 3-class sentiment macro-F1 of 0.4573** and **mBERT the best 6-class emotion macro-F1 of 0.2703**. The CNN and BiLSTM baselines come within 0.004 / 0.028 macro-F1 of the best transformer on each task, respectively - i.e. once a strong Urdu fastText embedding is available, the contextual-subword advantage on this corpus is modest. The Linear SVM achieves the highest *accuracy* on both tasks (0.878) but collapses to majority-class prediction on the rare emotions, which is exactly the failure mode the macro-F1 metric is designed to surface.

10.2 Limitations

- Labels remain weakly supervised; a small expert-annotated test set would tighten our metric ceiling.
- Domain is narrow (Pakistani Twitter); transfer to news or prose Urdu is not measured here.
- We do not yet apply self-training / pseudo-labelling on the ≈ 515 K rows with `Category = NaN`, which is the natural next step to exploit the full 1 M tweet count.

10.3 Future work

- Active learning on the most-uncertain `Category = NaN` rows to bootstrap a higher-quality 1 M training set.
- Adapter-based fine-tuning (LoRA, prefix-tuning) of XLM-R for parameter-efficient deployment.
- Cross-corpus evaluation against the Roman-Urdu sentiment benchmarks identified in Milestone 2.

11 References

IEEE-style references, drawn from the 20-paper review in Milestone 2.

1. M. T. Ali *et al.*, “SentiUrdu-1M: A large-scale weakly-labelled Urdu Twitter dataset,” *Data in Brief*, 2023. DOI: 10.17632/rz3xg97rm5.1.
2. A. Vaswani *et al.*, “Attention Is All You Need,” *NeurIPS*, 2017.
3. J. Devlin *et al.*, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” *NAACL*, 2019.
4. A. Conneau *et al.*, “Unsupervised Cross-Lingual Representation Learning at Scale,” *ACL*, 2020. (XLM-R)
5. Y. Liu *et al.*, “RoBERTa: A Robustly Optimized BERT Pretraining Approach,” *arXiv:1907.11692*, 2019.
6. Y. Kim, “Convolutional Neural Networks for Sentence Classification,” *EMNLP*, 2014.
7. S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
8. D. Bahdanau, K. Cho, Y. Bengio, “Neural Machine Translation by Jointly Learning to Align and Translate,” *ICLR*, 2015.
9. M. Schuster, K. K. Paliwal, “Bidirectional Recurrent Neural Networks,” *IEEE Trans. on Signal Processing*, 1997.
10. T. Mikolov *et al.*, “Advances in Pre-Training Distributed Word Representations,” *LREC*, 2018. (fastText)
11. T. Mikolov *et al.*, “Efficient Estimation of Word Representations in Vector Space,” *ICLR Workshop*, 2013. (word2vec)
12. J. Pennington, R. Socher, C. D. Manning, “GloVe: Global Vectors for Word Representation,” *EMNLP*, 2014.
13. F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *JMLR*, vol. 12, pp. 2825–2830, 2011.

14. T. Wolf *et al.*, “Transformers: State-of-the-Art Natural Language Processing,” *EMNLP Demos*, 2020.
15. I. Loshchilov, F. Hutter, “Decoupled Weight Decay Regularization,” *ICLR*, 2019. (AdamW)
16. D. P. Kingma, J. Ba, “Adam: A Method for Stochastic Optimization,” *ICLR*, 2015.
17. N. Micikevicius *et al.*, “Mixed Precision Training,” *ICLR*, 2018.
18. C. Cortes, V. Vapnik, “Support-Vector Networks,” *Machine Learning*, vol. 20, pp. 273–297, 1995.
19. J. Platt, “Probabilistic Outputs for Support Vector Machines and Comparisons to Regularized Likelihood Methods,” *Advances in Large-Margin Classifiers*, 1999.
20. G. Salton, C. Buckley, “Term-weighting approaches in automatic text retrieval,” *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988. (TF-IDF)

12 AI Usage Declaration

This declaration accompanies the present technical report in compliance with the course AI-use policy.

- **Tools used.** Anthropic Claude Code was used as a coding assistant for: (a) extracting the eight-step preprocessing pipeline from Milestone 1 into a reusable Python module; (b) drafting the Python code of the deep-learning trainer (`train_dl.py`), the transformer fine-tuning wrapper (`train_transformer.py`), and the notebook scaffolding for §§1–4 of the implementation and evaluation notebooks; (c) generating the first draft of this report and of `04_architecture_math.md`.
- **Human review.** All AI-generated code and prose were reviewed, executed, and modified by the authors. The final implementation choices (label canonicalisation, class-weighted loss, fp16 training, emoji-removal preprocessing) are the authors’ decisions, taken in consultation with the AI assistant.
- **No undisclosed data.** No private or unpublished data was shared with the AI tool. The only inputs were (a) public source code we authored and (b) the structure of the public SentiUrdu-1M dataset.

13 Contribution Statement

Author	NUM ID	Primary contributions
Raqib Hayat	NUM-BSCS-2022-40	Dataset analysis notebook, mathematical modelling document, technical report writing, training curve interpretation.
Abu Bakar	NUM-BSCS-2022-41	Preprocessing module, deep-learning + transformer implementation, evaluation notebook, hyperparameter sweep, reproducibility plumbing.

Both authors reviewed and signed off on the final report and the released code.